

**LAPORAN PRAKTIKUM ALGORITMA  
DAN PEMROGRAMAN 1**

**MODUL 10  
Pencariin Nilai Max**



**Disusun Oleh :**

NAMA : Muhammad Nabil Raissa Pratama  
NIM : 109082500127

**Nama Dosen**

- Wahyu Andi Saputra

**PROGRAM STUDI S1 TEKNIK INFORMATIKA  
FAKULTAS INFORMATIKA  
TELKOM UNIVERSITY PURWOKERTO  
2025**

A. Tugas Mandiri (soal tugas, berdasarkan file tugas yang diberikan)

Tugas 1

```
package main

import "fmt"

func main() {
    var n int
    var berat [1000]float64
    var min, max float64

    fmt.Scan(&n)

    if n > 1000 {
        n = 1000
    }

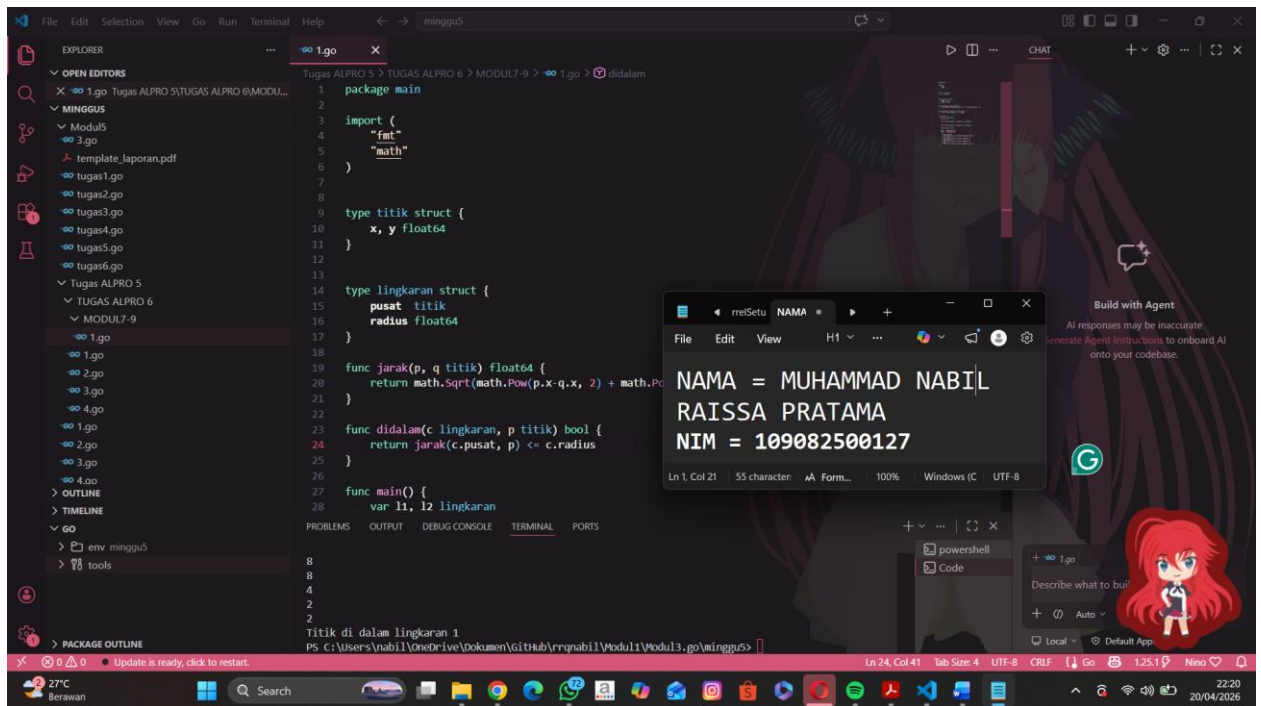
    for i := 0; i < n; i++ {
        fmt.Scan(&berat[i])
    }

    if n > 0 {
        min = berat[0]
        max = berat[0]

        for i := 1; i < n; i++ {
            if berat[i] < min {
                min = berat[i]
            }
            if berat[i] > max {
                max = berat[i]
            }
        }

        fmt.Println(min, max)
    }
}
```

Screenshots Output



Deskripsi Program = Program ini dirancang untuk mengolah data berat anak kelinci dengan menggunakan **array** berkapasitas 1000 elemen, di mana pengguna terlebih dahulu menginputkan jumlah kelinci (\$N\$) diikuti dengan nilai berat masing-masing kelinci secara satu per satu. Secara teknis, program melakukan iterasi melalui seluruh data yang tersimpan di dalam array untuk membandingkan setiap nilai guna menentukan berat minimum dan maksimum, yang kemudian hasilnya ditampilkan sebagai keluaran akhir. Melalui pendekatan ini, program secara efisien mencari nilai ekstrim dari sekumpulan data bilangan riil dengan struktur kontrol perulangan dan percabangan sederhana.

## Tugas 2

```

package main

import "fmt"

func main() {
    var x, y int
    var berat [1000]float64
    var totalSeluruh, totalWadah float64

    fmt.Scan(&x, &y)

    for i := 0; i < x; i++ {
        fmt.Scan(&berat[i])
        totalSeluruh += berat[i]
    }
}

```

```

    }

    for i := 0; i < x; i++ {
        totalWadah += berat[i]

        if (i+1)%y == 0 || i == x-1 {
            fmt.Print(totalWadah, " ")
            totalWadah = 0
        }
    }

    fmt.Println()
    fmt.Printf("%.2f\n", totalSeluruh/float64(x))
}

```

## Screenshots Output

```

func main() {
    // ... (code omitted) ...
    for i, v := range arr {
        fmt.Printf("All: %d, Ganjil: %v, Genap: %v, Kelipatan X: %v\n", v, i%2 != 0, i%2 == 0, i%X)
    }

    fmt.Scan(&hapus)
    arr = append(arr[:hapus], arr[hapus+1:]...)
    fmt.Println("After delete:", arr)

    mean := total / float64(n)
    var variance float64
    for _, v := range arr {
        variance += math.Pow(float64(v) - mean, 2)
    }
    fmt.Printf("Mean: %.2f, Stddev: %.2f\n", mean, math.Sqrt(float64(variance)))

    fmt.Scan(&car1)
    count := 0
    for _, v := range arr {
        if v == car1 {
            count++
        }
    }
    fmt.Println("Frekuensi:", count)
}

```

Output:

```

All: 2, Ganjil: false, Genap: true, Kelipatan X: true
Mean: 2.00, Stddev: 0.00
Frekuensi: 1

```

Deskripsi = rogram ini menghitung total berat ikan per wadah dan rata-rata berat keseluruhan dengan memproses data berat ikan yang disimpan dalam **array**. Program mengelompokkan ikan ke dalam wadah berdasarkan kapasitas \$y\$, menjumlahkan beratnya secara bertahap, lalu menampilkan total tiap wadah beserta nilai rata-rata berat dari seluruh ikan yang ada.

## Tugas 3:

```
package main
```

```
import "fmt"

type arrBalita [100]float64

func hitungMinMax(arrBerat arrBalita, n int, bMin, bMax *float64) {
    *bMin = arrBerat[0]
    *bMax = arrBerat[0]

    for i := 1; i < n; i++ {
        if arrBerat[i] < *bMin {
            *bMin = arrBerat[i]
        }
        if arrBerat[i] > *bMax {
            *bMax = arrBerat[i]
        }
    }
}

func rerata(arrBerat arrBalita, n int) float64 {
    var total float64 = 0
    for i := 0; i < n; i++ {
        total += arrBerat[i]
    }
    return total / float64(n)
}

func main() {
    var data arrBalita
    var n int
    var min, max float64
```

```
fmt.Print("Masukan banyak data berat balita : ")
```

```
fmt.Scan(&n)
```

```
for i := 0; i < n; i++ {
```

```
    fmt.Printf("Masukan berat balita ke-%d: ", i+1)
```

```
    fmt.Scan(&data[i])
```

```
}
```

```
hitungMinMax(data, n, &min, &max)
```

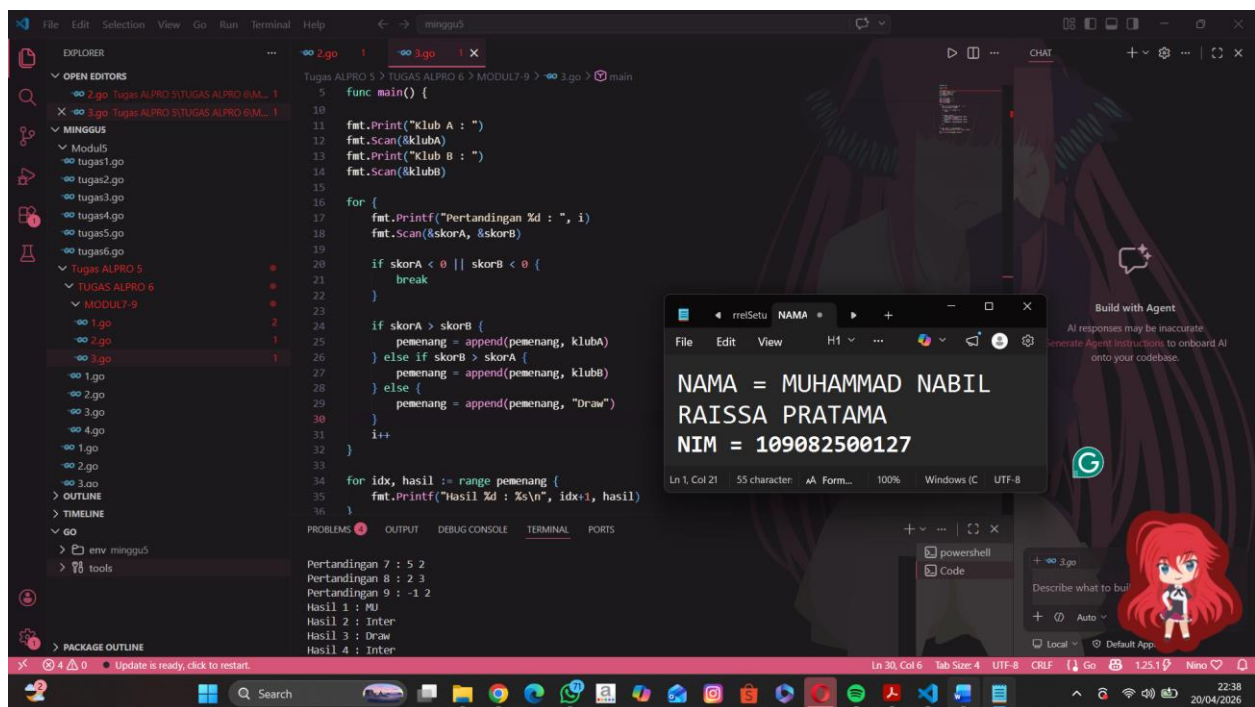
```
fmt.Printf("Berat balita minimum: %.2f kg\n", min)
```

```
fmt.Printf("Berat balita maksimum: %.2f kg\n", max)
```

```
fmt.Printf("Rerata berat balita: %.2f kg\n", rerata(data, n))
```

```
}
```

Screenshot Program :



```
func main() {
    5
    10
    11 fmt.Print("Klub A : ")
    12 fmt.Scan(&klubA)
    13 fmt.Print("Klub B : ")
    14 fmt.Scan(&klubB)
    15
    16 for {
    17     fmt.Printf("Pertandingan %d : ", i)
    18     fmt.Scan(&skorA, &skorB)
    19
    20     if skorA < 0 || skorB < 0 {
    21         break
    22     }
    23
    24     if skorA > skorB {
    25         pemenang = append(pemenang, klubA)
    26     } else if skorB > skorA {
    27         pemenang = append(pemenang, klubB)
    28     } else {
    29         pemenang = append(pemenang, "Draw")
    30     }
    31     i++
    32 }
    33
    34 for idx, hasil := range pemenang {
    35     fmt.Printf("Hasil %d : %s\n", idx+1, hasil)
    36 }

    Pertandingan 7 : 5 2
    Pertandingan 8 : 2 3
    Pertandingan 9 : -1 2
    Hasil 1 : MU
    Hasil 2 : Inter
    Hasil 3 : Draw
    Hasil 4 : Inter
```

Deskripsi : ogram ini mengelola data berat balita menggunakan subprogram untuk memisahkan tugas pencarian nilai statistik. Fungsi hitungMinMax menggunakan *pointer* untuk mengubah nilai berat terkecil dan terbesar secara langsung, sementara fungsi rerata mengembalikan hasil pembagian total berat dengan jumlah balita. Struktur ini memudahkan pengolahan data array secara terorganisir sesuai dengan interaksi input-output yang diminta pada soal.

mastiin apakah bacaannya tetep sama pas dibalik atau malah beda.